2023 1st IEEE International Conference on Smart Technology (ICE-SMARTec).
Bandung, Indonesia, 17-19 July 2023

Development of Convolutional Neural Network (CNN) Method for Classification of Tomato Leaf Disease Based on Android

Otto Nathanael
Dept. of Computer Engineering, Harapan Bangsa Institute of Technology
Bandung, Indonesia
sk-19005@students.ithb.ac.id

Maclaurin Hutagalung
Dept. of Computer Engineering, Harapan Bangsa Institute of Technology
Bandung, Indonesia
maclaurin@ithb.ac.id

Yoyok Gamaliel
Dept. of Computer Engineering, Harapan Bangsa Institute of Technology
Bandung, Indonesia
yoyok@ithb.ac.id

2023 1st IEEE International Conference on Smart Technology (ICE-SMARTec) | 979-8-3503-0615-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICE-SMARTECH59237.2023.10461962

***Abstract*—Tomato has become one of the vegetable fruit plants that Indonesian people widely cultivate. In cultivation activities, tomato cultivators will face diseases and pests that affect growth, such as diseases on tomato leaves. With multiple types of tomato leaf diseases, it is challenging for a cultivator to distinguish between them. This research aims to provide cultivators with valuable information about tomato leaf diseases and how to address them using Android-based disease detection. The tomato leaf disease detection application was developed on the Android platform that uses the camera as its main feature to be widely used by the community. TensorFlow Lite is developed as a model for tomato plant disease detection by implementing the Convolutional Neural Network (CNN) method with a total of 9,000 sick and 1,000 healthy tomato leaf datasets. The accuracy, loss, and confidence score are calculated. The system was tested using 50 Epochs, with a dataset comparison scenario of 80:10:10 when training the model. The results obtained by performing five tests show an average accuracy of 92.31% and an average loss of 24.81%, with a confidence score of 80-95%.**

Keywords—Tomato Plants, Tomato Disease, Android Application, Convolutional Neural Network, CNN

I. INTRODUCTION

According to Badan Pusat Statistik (2021), Indonesia can produce 1,114,399 tons of tomatoes per year, where 26.23% is produced in West Java, with the most significant production reaching 292,309 tons [1]. Tomato growers prioritize the health of their tomato plants to avoid the risk of crop failure due to bad weather [2] or diseases such as late blight, which can damage tomato fruit [3]. However, a lack of knowledge about plant diseases can increase the risk of crop failure [4], and other factors can lead to decreasing nutritional quality and producing poor-quality seeds. However, looking for solutions or anticipating crop failure through social media platforms is worthless because of the truth of the information and time effectiveness.

The current solutions have yet to fully help tomato cultivators because they are not ready for use in various conditions. This is an opportunity to develop new applications that are superior and provide more targeted benefits. This application is designed to help tomato cultivators prevent, control, and treat plant diseases, as well as help increase crop yields.

To overcome the problem of tomato crop failure caused by diseases such as brown spotting on tomato leaves and yellowing of leaves, a mobile application was created with the feature of classifying tomato leave using the Convolutional Neural Network (CNN) method, which can automatically detect important features of each image without human assistance [5]. Adding data augmentation to pre-trained that believe to improve the results of the classification [6].

From the existing problems, the idea was to create a mobile application that allows tomato cultivators to monitor the conditions of tomato plants by taking pictures of diseased leaves and displaying the name of the disease, the definition of the disease, and the solution in text form.

In addition, this application also provides information about the types of diseases that are commonly found in tomato plants. With this application, cultivators can find out the health condition of tomato leaves and how to deal with the disease.

II. METHOD

Developing a tomato plant leaf disease detection system focuses on three processes: how tomatoes can get a disease, making CNN models, and making applications on Android. The application will be designed and developed using a machine-learning model that implements image classification techniques.

A. Tomato plant disease

Tomato plants are said to be diseased if their growth deviates from normal conditions. The causes consist of several kinds, including fungi, bacteria, and viruses. Signs of disease in tomato plants are characterized by damage to tomato leaves and stunted plant growth [7], [8].

B. Design and Implementation of CNN Model

The dataset used in this research consists of images collected from a public data science learning site/platform, Kaggle [9]. The total dataset used is 10,000 images classified into nine types of disease and one healthy. The specifics of the dataset consisting of the tested dataset and trained dataset are presented in Table I. The dataset will be divided into three parts training, testing, and prediction.

Model creation uses Python as the primary programming language and is equipped with the Tensorflow framework and

Keras as a package from Tensorflow [10]. Keras is one of the modules developed by Google to make it easier for users to build neural network models with several layers and different coatings. The process of system design stages can be illustrated in Fig. 1.

TABLE I. DETAILS OF TOMATO DISEASES IN THE DATASET

No	Disease Name	Dataset
1	Tomato Bacterial spot	1000
2	Tomato Early blight	1000
3	Tomato Healthy	1000
4	Tomato Late blight	1000
5	Tomato Leaf Mold	1000
6	Tomato Septoria leaf spot	1000
7	Tomato Spider mites Two-spotted spider mite	1000
8	Tomato Target Spot	1000
9	Tomato Mosaic virus	1000
10	Tomato Yellow Leaf Curl Virus	1000

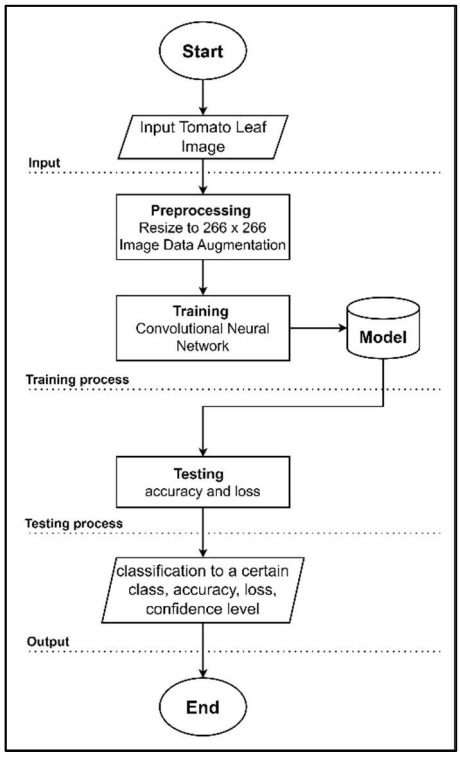


Fig. 1. CNN Design System

The model that has been trained will be used for the process of making predictions. From training data that has been split into three parts, the image data will be processed first before entering the feature learning stage. This preprocessing is a preparation for processing images to be used as datasets by resizing and rescaling the images [6] [11]. Preprocessing is carried out for training, testing, and predictive data. Preprocessing for the dataset is done only once before the dataset is used as a reference by the system for further use in testing the system.

The feature learning stage explains how images past the preprocessing stage are processed in feature learning. The process that occurs in this section is to "encode" an image into a feature that consists of numbers that represent an image. Images that have passed the preprocessing stage will be sampled and split into several small parts representing all objects from the whole picture.

The feature learning stage consists of 2 main parts: the convolution layer and the pooling layer. The convolution layer performs a pixel extension operation on the filtered image. The filter in question is a matrix that can be changed from the original image to a feature map. This filter is often referred to as a feature detector. The actual form in its application is a square matrix (the same number of rows and columns). The 3×3 filter is the most widely used for general training and CNN. The filter size is adjusted according to the researcher's needs when designing the model. It is essential to choose a filter size that does not exceed the size of the image to be captured, so the filter must be smaller to apply to the image. The convolution process is repeated for each pixel value in the image so that the image size becomes smaller than the original size.

When resizing the original image, some information may be lost due to the smaller size. However, the main goal of a feature detector is to find important features in an integral image. The detector feature speeds up image processing because fewer data pixels must be processed. After combining the image with the kernel, the results of adding pixel by pixel are added for the visualization process. In Fig. 2, the white part shows the transformed image, with the kernel moving from the top left corner to the bottom right corner. The convolution results of these images produce a new matrix, as shown in Fig. 3.

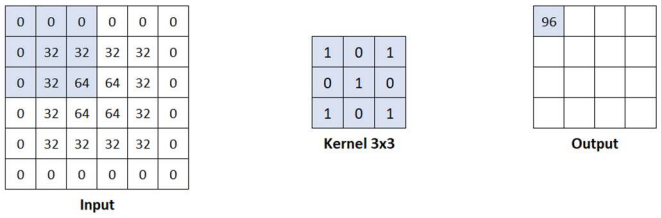


Fig. 2. Initial Convolution Process

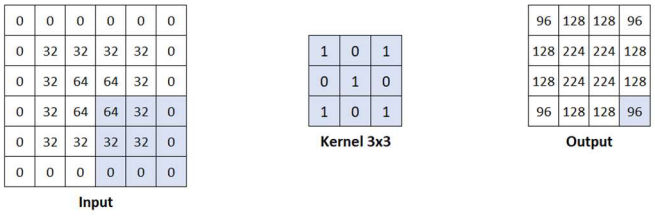


Fig. 3. Final convolution process

In the convolution process, activation is carried out using the Rectified Linear Units (ReLU) [12] activation function to reduce the linear properties of the feature map in the convolution layer. We need the ReLU function because the image has a non-linear trend. Non-linear means no linear relationship exists between elements in the image, for example, a tomato leaf with a leaf background.

The next step is to do the pooling layer process. In this research, the type of pooling used is max pooling layer. As with convolution, at this stage, we create a new layer size of 2x2. Max pooling will choose the highest value from all the values contained in it. The function of layer pooling is to reduce the dimensions of the feature map (decreasing resolution), which will speed up the computation process because there are fewer parameters to process.

The result of the feature maps will be convoluted again to produce new smaller features so that the detail of the features increases. Images with high resolution can consist of

thousands of pixels. Therefore, image processing is done repeatedly to more accurately represent the features of the object you want to detect. The more detailed the pixel value of the convolved feature results, the easier it is to guess the high value as one of the objects you want to classify.

Flatten is used to change the polling layer matrix into a single vector. This vector will be part of the input layer. To do this, the rows of the pooled features are taken alternately and combined into a single column. All poll layers will be converted into one vector. The alignment process can be seen in Fig. 4.

The fully-connected layer is the last to be processed in the neural network and is used in the image classification process as readable input. The fully-connected layer consists of neurons connected with all the neurons of the previous layer, as happens in an artificial neural network. The purpose of using a fully-connected layer in the Perceptron Multi-layer method is to process data so that it can be classified. The difference between the fully-connected and convolution layers is that the neurons are only connected to certain input areas in the convolution layer. In contrast, the neurons are connected in the fully-connected layer. The two layers still use dot products, and their functions do not differ significantly. All processes can also be visualized in Fig. 5, and the comprehensive mapping of model architecture is in Table II.

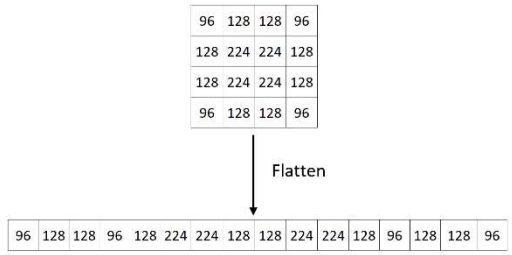


Fig. 4. Flatten Layer Process

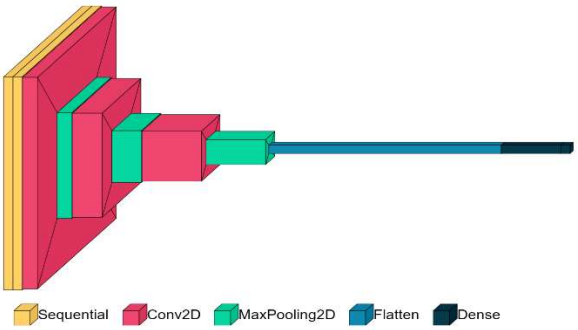


Fig. 5. Convolutional Neural Network Architecture Visualization

The number of parameters in a convolutional neural network refers to the total number of learnable weights and biases used in the network. These include the weights and biases of the convolutional layers, fully connected layers, and any other layers with learnable parameters. Each layer in a CNN has its own set of weights and biases learned during the training process. In a convolutional layer and dense layer, the number of parameters is determined in Eq. 1 [13] and Eq.2 [13].

The number of parameters for all MaxPooling2D layers is all 0. The reason is that this layer doesn't learn anything, but it does is finding the maximum numbers for each 2 x 2 pool will help to simplify the model and extract local features. Max pooling layer is determined in Eq.3 [13].

TABLE II. MAPPING OF MODEL ARCHITECTURAL PROCESS STAGE

Layer	Output Shape	Parameter
sequential (Sequential)	(16, 256, 256, 3)	0
sequential_1 (Sequential)	(16, 256, 256, 3)	0
conv2d (Conv2D)	(16, 254, 254, 64)	1792
max_pooling2d (MaxPooling2D)	(16, 127, 127, 64)	0
conv2d_1 (Conv2D)	(16, 125, 125, 32)	18464
max_pooling2d_1 (MaxPooling2D)	(16, 62, 62, 32)	
conv2d_2 (Conv2D)	(16, 60, 60, 16)	4624
max_pooling2d_2 (MaxPooling2D)	(16, 30, 30, 16)	0
flatten (Flatten)	(16, 14400)	0
dense (Dense)	(16, 64)	921664
dense_1 (Dense)	(16, 10)	650
Total parameter:		947,194

$$\begin{aligned} Wc &= K^2 \, x \, C \, x \, N \\ Bc &= N \\ Pc &= Wc + Bc \end{aligned} \tag{1}$$

Where Wc represents a number of weights of the conv layer, Bc represents a number of biases of the conv layer, Pc represents a number of parameters of the conv layer. K is the size of the kernel used in the conv layer, N is the number of kernels, and C is the number of channels of the input image.

$$\begin{aligned} Wfc &= Fp \, x \, F \\ Bf &= F \\ Pf &= Wfc \; + \; Bf \end{aligned} \tag{2}$$

Where Wfc represents the number of weights of the fully connected layer, Bf represents the number of biases of the fully connected layer, Pf represents the number of parameters of the fully connected layer. Fp represents the number of neurons in the previous fully connected layer. F represents the number of neurons in the fully connected layer.

$$O = \frac{I-P}{S} + 1 \tag{3}$$

Where O represent size of output image, I represents size of input image, Ps represents a pool size, and S represents a stride of the convolution operation.

After creating the CNN model, the model will be converted to a Tensorflow Lite model [14] that will later be used in Android applications.

C. *Building Android Application*

Android Studio is an integrated development environment (IDE) for developing the Android platform base on Kotlin programing language [15]. This application aims to help users use tomato plant disease detection based on the CNN method. SQLite database is used to store data containing the name of the disease, an explanation of the disease, the treatment for the disease, and history. Users will select the features in the application. For the tomato scanning feature, the user can input an image from the gallery or take a picture directly using the camera. During the detection process, it will load the Tensorflow Lite model that has been created. SQLite database id used for After the Tensorflow

Lite model provides identification results, the system retrieves data from the SQLite database for descriptions and solutions. The result is an image and text displayed on the Android, as shown in Fig. 6.

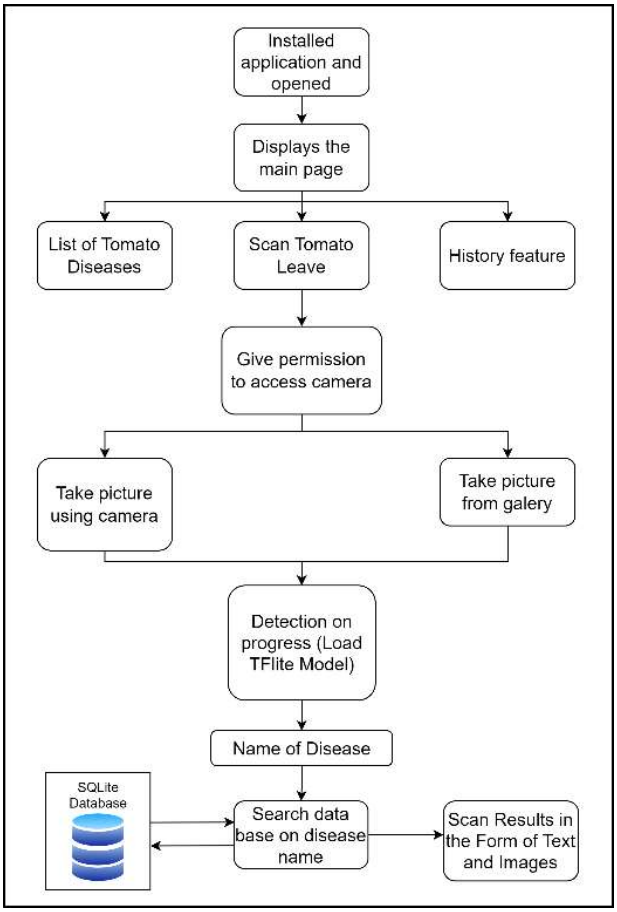


Fig. 6. Android Application Flow

III. Result And Discussion

Testing is used to measure the accuracy and loss of a model. Accuracy measures how effectively the model predicts the class of a given input image. At the same time, loss quantifies how well the model can reduce the difference between the expected and actual output. The training process minimizes the value of the loss function, and the confidence score is the likelihood that the model assigns to a specific prediction. A CNN will produce a probability distribution over the various possible classes for a given input.

The dataset is tested using two scenarios, 80:10:10 and 70:15:15, using 10, 20, 30, 40, and 50 epochs. This test is conducted to see the best epoch value based on which dataset ratio produces the optimum accuracy value of all tests. Tests were carried out five times for each epoch. All average accuracy and lost test are in Tabel III and Tabel IV.

From the Table III, it is known that the 80:10:10 dataset gets better accuracy by using 50 epochs compared to Table IV (70:15:15). Fig. 7 and Fig. 8 represent training accuracy and loss from the 80:10:10 ratio dataset with 50 epochs.

A test is carried out from the existing results by adding data augmentation at the preprocessing stage to find the best predictive results. The data augmentation added is random flip, random rotation, random zoom, and random translation. The results of data augmentation testing are shown in Table V.

TABLE III. 80:10:10 RATIO DATASET TO LOSS AND ACCURACY BASED ON EPOCH

Epoch	Loss	Validation	Accuracy (%)
10	0. 5185	0. 8268	82.68
20	0. 2957	0. 9026	90.26
30	0. 5719	0. 8402	84.02
40	0. 3384	0. 9076	90.76
50	0. 2733	0. 9242	92.42

TABLE IV. 70:15:15 RATIO DATASET TO LOSS AND ACCURACY BASED ON EPOCH

Epoch	Loss	Validation	Accuracy (%)
10	0. 4369	0. 8518	85.18
20	0. 3131	0. 8975	89.75
30	0. 2528	0. 9170	91.70
40	0. 2863	0. 9107	91.07
50	0. 2481	0. 9231	92.31

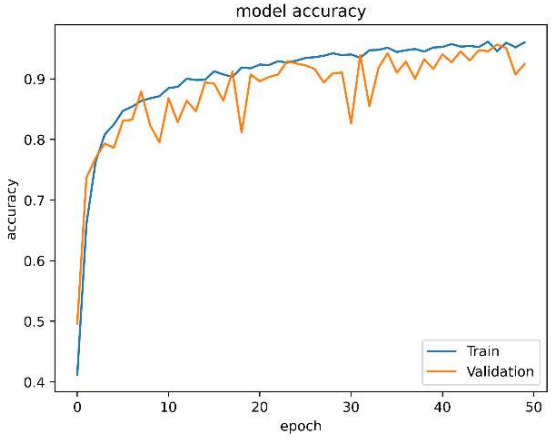


Fig. 7. Accuracy 80:10:10 Dataset Ratio With 50 Epoch

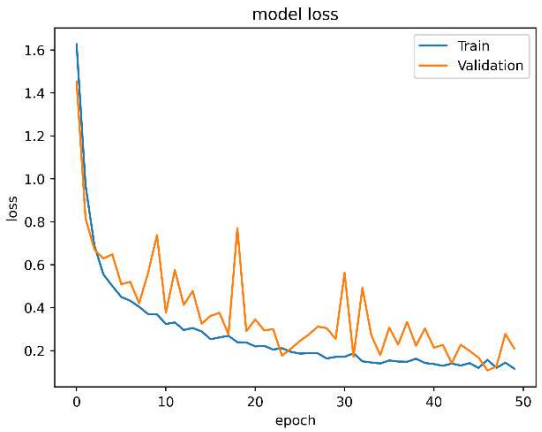
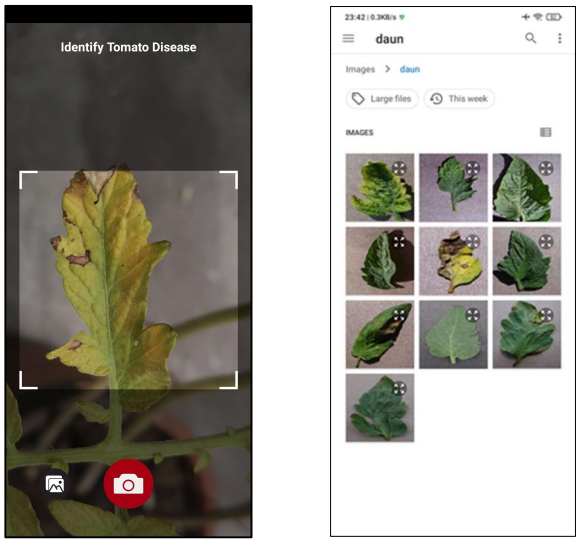


Fig. 8. Loss 80:10:10 Dataset Ratio With 50 Epoch

Model testing was carried out on the Android application to see the performance of the CNN model. Testing is done by entering images through a live camera and a gallery which can be seen in Fig. 9, and then looking at the confidence of the prediction results in Fig. 10. 10 types of disease were also tested on the Android application to determine the level of confidence in CNN predictions as shown in Tabel VI.

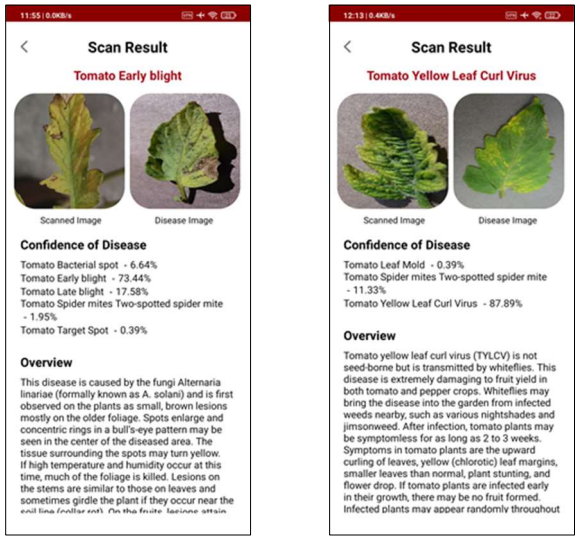
TABLE V. DATA AUGMENTATION TESTING

Data augmentation	Loss	Validation	Accuracy (%)
No augmentation	0.4069	0.9633	96.33%
Flip	0.1207	0.9702	97.02%
Rotation	0.1704	0.9633	96.33%
Zoom	0.1225	0.9772	97.72%
Translation	0.0951	0.9663	96.63%
Flip + Rotation	0.1621	0.9484	94.84%
Flip + Zoom	0.0878	0.9772	97.72%
Flip + Translation	0.1264	0.9514	95.14%
Rotation + Zoom	0.2637	0.9315	93.15%
Rotation + Translation	0.3311	0.9008	90.08%
Zoom + Translation	0.2067	0.9276	92.75%
Flip + Rotation + Zoom	0.3493	0.8899	88.99%
Flip + Rotation + Translation	0.5271	0.8442	84.42%
Flip + Zoom + Translation	0.3686	0.8879	88.79%
Rotation + Zoom + Translation	0.2975	0.9097	90.97%
Flip + Rotation + Zoom + Translation	0.7020	0.8442	84.42%



(a) (b)

Fig. 9. (a) Picture From The Camera, (b) Picture From The Gallery



(a) (b)

Fig. 10. (a) Test Results From The Camera, (b) Test Results From The Gallery

TABLE VI. CONFIDENCE SCORE IN ANDROID APPLICATION

No	Disease Name	Confidence
1	Tomato Bacterial spot	95.7%
2	Tomato Early blight	90.62%
3	Tomato Healthy	90.23%
4	Tomato Late blight	94.92%
5	Tomato Leaf Mold	84.77%
6	Tomato Septoria leaf spot	80.47%
7	Tomato Spider mites Two-spotted spider mite	96.09%
8	Tomato Target Spot	87.11%
9	Tomato Mosaic virus	80.47%
10	Tomato Yellow Leaf Curl Virus	90.62%

IV. CONCLUSION

Based on the testing results, it can be concluded that using a ratio of 80:10:10 with 50 epochs and data augmentation random flip and random zoom is the most appropriate because it gets the highest accuracy 97.72% with the lowest loss 8.78%. In addition, the built system can recognize and classify image type labels through testing on an image taken from an Android application. This test has been successfully carried out separately from the training process. After conducting tests using real leaf images from photographs, the model's performance was good enough to recognize the types of diseases on tomato leaves with a confidence score of 80-95%. However, the leaf background determines the accuracy of the classification process, so there are still some errors in the classification process of tomato leaf images because an unclean background will make mistakes during the classification process.

REFERENCES

[1] B. P. Statistik, “Produksi Tanaman Sayuran 2021,” [Online]. Available: https://www.bps.go.id/indicator/55/61/1/produksi-tanaman-sayuran.html.

[2] S. M, H. E. Saputra dan R. Hermanto, dalam *Bertanam tomat di musim hujan*, Bogor, Penebar Swadaya, 2015, p. 84.

[3] H. G. S. Gate, “5 Jamur yang Sering Menyerang Tanaman Tomat, Bikin Buah Membusuk,” KOMPAS.com, 24 Mei 2021. [Online]. Available: https://www.corteva.id/berita/5-Jamur-yang-Sering-Menyerang-Tanaman-Tomat-Bikin-Buah-Membusuk.html.

[4] F. M. Taufiq, “Perlunya Akses Informasi Untuk Mendukung Usaha Tani,” 3 Februari 2021. [Online]. Available: https://diskominfo.acehtengahkab.go.id/berita/kategori/teknologi/perlunya-akses-informasi-untuk-mendukung-usaha-tani.

[5] J. Gu, Z. Wang, J. Kuen, L. Ma, A. Shahroudy, B. Shuai, T. Liu, X. Wang, G. Wang, J. Cai and T. Chen, "Recent advances in convolutional neural networks," Pattern Recognition, vol.77, pp. 354-377, 2018.

[6] A. Mikolajczyk and M. Grochowski, "Data augmentation for improving deep learning in image classification problem," *International Interdisciplinary PhD Workshop (IIPhDW),* pp. 117-122, 2018.

[7] V. K. Singh, A. K. Singh and A. Kumar, "Disease management of tomato through PGPB: current trends and future perspective," *CrossMark,* 2017.

[8] U. W. Kusuma, N. Azizah and R. Widodo, "SISTEM PAKAR DIAGNOSA PENYAKIT TANAMAN TOMAT MENGGUNAKAN METODE FORWARD CHAINING," *Nusantara of Engineering,* vol. 3, no.2, 2016.

[9] "Kaggle," [Online]. Available: https://www.kaggle.com/. [Accessed February 2023].

[10] "Tensorflow," [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras. [Accessed February 2023].

[11] A. Krizhevsky, I. Sutkever and G. E. hinton, "ImageNet Classification with Deep Convolutional Neural Networks", Advances in Neural Information Processing Systems, 2012.

[12] L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma5,, j. Santamaría, M. A. Fadhel, M. Al-Amidie and L. Farhan, "Review of deep learning: concepts, CNN architectures, challenges, applications, future directions," *Journal of big Data,* 2021.

[13] S. Mallick and S. Nayak, "Number of Parameters and Tensor Sizes in a Convolutional Neural Network (CNN)," 22 May 2018. [Online]. Available: https://learnopencv.com/number-of-parameters-and-tensor-sizes-in-convolutional-neural-network. [Accessed February 2023].

[14] "Tensorflow Lite," Google, [Online]. Available: https://www.tensorflow.org/lite. [Accessed February 2023].

[15] "Developer Android," [Online]. Available: https://developer.android.com/kotlin. [Accessed February 2023].